Introduction to Database Systems

2024-Fall

6. Database Design

6.1 Data Dependency and Normalization of Relational Schema

- Some dependent relations exist between attributes.
- ***Function dependency (FD):*** the most basic kind of data dependencies. The value of one or a group attributes can ***decide*** the value of other attributes.

StudentGrades: < StudentID , CourseID, Instructor, Grade>

FD: StudentID, CourseID -> Grade

CourseID -> Instructor

StudentID, CourseID -> Instructor

FD is the most important in general database design.

Function Dependency

- ***Fully Functional Dependency :***
  If X and Y are an attribute set of a relation, Y is fully functional dependent on X, if Y is functionally dependent on X but not on any proper subset of X.
- ***Partial Functional Dependency :***
  A functional dependency X->Y is a partial dependency if Y is functionally dependent on X and Y can be determined by any proper subset of X.
  - For example, we have a relationship AC->B, A->D, and D->B.

Multi-valued Dependency

- ***Multi-valued Dependency (MVD):*** the value of some attribute can decide a group of values of some other attributes.

StudentActivities:< StudentID, Hobby,Course>

StudentID ->> Hobby

StudentID ->> Course

StudentID	Hobby	Course
1	Music	Math
1	Music	Science
1	Painting	Math
1	Painting	Science

Join Dependency

- ***Join Dependency (JD):*** the constraint of lossless join decomposition.
- 在关系数据库中，给定一个关系R及其属性的一个分解集合{R1, R2, ..., Rn}，如果R等于这些子关系的自然连接结果，则称存在一个连接依赖，记作 *(R1, R2, ..., Rn)。

Projects

ProjectID	EmployeeID	Skill
1	A	Java
1	B	Python
1	A	SQL
2	B	Java

ProjectEmployees

ProjectID	EmployeeID
1	A
1	B
2	B

EmployeeSkills

EmployeeID	Skill
A	Java
A	SQL
B	Python
B	Java

ProjectSkills

ProjectID	Skill
1	Java
1	Python
1	SQL
2	Java

这些关系的分解满足连接依赖 *(ProjectEmployees, EmployeeSkills, ProjectSkills)，因为通过自然连接操作，可以重构出原始的 Projects 关系

Features of Good Relational Designs

- Suppose we combine *instructor* and *department* into *in_dep,* which represents the natural join on the relations *instructor* and *department*

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- There is repetition of information
- Need to use null values (if we add a new department with no instructors)

The Evils of Redundancy

- ***Redundancy*** is at the root of several problems associated with relational schemas:
  - redundant storage, insert/delete/update anomalies
- Integrity constraints, in particular *functional dependencies*, can be used to identify schemas with such problems and to suggest refinements.
- Main refinement technique: ***decomposition*** (replacing ABCD with, say, AB and BCD, or ACD and ABD).
- Decomposition should be used judiciously:
  - Is there reason to decompose a relation?
  - What problems (if any) does the decomposition cause?

Functional Dependencies (FDs)

- A functional dependency X → Y holds over relation R if, for every allowable instance *r* of R:
  - $t1 \in r, t2 \in r, \pi X(t1) = \pi X(t2)$ implies $\pi Y(t1) = \pi Y(t2)$
  - i.e., given two tuples in *r*, if the X values agree, then the Y values must also agree. (X and Y are *sets* of attributes.)
- An FD is a statement about ***all*** allowable relations.
  - ***Must be identified based on semantics of application***.
  - Given some allowable instance *r1* of R, we can check if it violates some FD *f*, but we cannot tell if *f* holds over R!
- E.g. K is a candidate key for R means that K→ R
  - However, K→R does not require K to be *minimal*!

Keys and Functional Dependencies

ID	*name*	*salary*	*dept_name*	*building*	*budget*

- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

  in_dep (*ID, name, salary, dept_name, building, budget*).

  We expect these functional dependencies to hold:

  dept_name→ *building*

  ID → *building*

  but would not expect the following to hold:

  dept_name → *salary*

Trivial Functional Dependencies 平凡函数依赖

ID	*name*	*salary*	*dept_name*	*building*	*budget*

- **In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$**
  Example*:*
  - *ID, name* $\rightarrow$ *ID*
  - *name* $\rightarrow$ *name*

Example: Constraints on Entity Set

- Consider relation obtained from Hourly_Emps:
  - Hourly_Emps (*ssn, name, lot, rating, hrly_wages*, *hrs_worked*)
- *Notation*: We will denote this relation schema by listing the attributes: SNLRWH
  - This is really the *set* of attributes {S,N,L,R,W,H}.
  - Sometimes, we will refer to all attributes of a relation by using the relation name. (e.g., Hourly_Emps for SNLRWH)
- Some FDs on Hourly_Emps:
  - *ssn* is the key: S → SNLRWH
  - *rating* determines *hrly_wages*: R → W

Example (Contd.)

- Problems due to R →W :
- *Update anomaly*: Can we change W in just the 1st tuple of SNLRWH?
- *Insertion anomaly*: What if we want to insert an employee and don't know the hourly wage for his rating?
- *Deletion anomaly*: If we delete all employees with rating 5, we lose the information about the wage for rating 5!

Wages

R	W
8	10
5	7

Hourly_Emps2

S	N	L	R	H
123-22-3666	Attishoo	48	8	40
231-31-5368	Smiley	22	8	30
131-24-3650	Smethurst	35	5	30
434-26-3751	Guldu	35	5	32
612-67-4134	Madayan	35	8	40

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

Reasoning About FDs

- Given some FDs, we can usually infer additional FDs:
  - *ssn*→ *did*, *did* → *lot* implies *ssn*→ *lot*
- An FD *f* is *implied by* a set of FDs *F* if *f* holds whenever all FDs in *F* hold.
  - F+ = *closure of F* is the set of all FDs that are implied by *F*.
- Armstrong's Axioms (X, Y, Z are sets of attributes):
  - *Reflexivity*: If X⊆Y, then Y→X
  - *Augmentation*: If X→Y, then XZ→YZ for any Z
  - *Transitivity*: If X→Y and Y→Z, then X→Z
- These are *sound* and *complete* inference rules for FDs!

Reasoning About FDs (Contd.)

- Couple of additional rules (that follow from AA):
  - ***Union***: If X → Y and X → Z, then X → YZ
  - ***Decomposition***: If X →YZ, then X→Y and X→Z
- Example: Contracts(*cid, sid, jid, did, pid, qty, value*), and:
  - C is the key: C→CSJDPQV
  - Project purchases each part using single contract: JP → C
  - Dept purchases at most one part from a supplier: SD → P
- JP → C, C → CSJDPQV imply JP → CSJDPQV SD→P implies SDJ→ JP SDJ → JP, JP→ CSJDPQV imply SDJ → CSJDPQV

Reasoning About FDs (Contd.)

- Computing the ***closure*** of a set of FDs can be expensive. (Size of closure is exponential in # attrs!)
- Typically, we just want to check if a given FD *X→ Y* is in the closure of a set of FDs *F*. An efficient check:
  - Compute *attribute closure* of X (denoted X+) wrt *F:*
    - Set of all attributes A such that X→ A is in F+
    - There is a linear time algorithm to compute this.
  - Check if Y is in X+
- Does F={A→B, B→C, CD→E} imply A→E?
  - i.e, is A→ E in the closure F+? Equivalently, is E in A+ ?

Normal Forms

- Returning to the issue of schema refinement, the first question to ask is whether any refinement is needed!
- If a relation is in a certain *normal form* (BCNF, 3NF etc.), it is known that certain kinds of problems are avoided/minimized. This can be used to help us decide whether decomposing the relation will help.
- ***Role of FDs in detecting redundancy***:
  - Consider a relation R with 3 attributes, ABC.
    - No FDs hold: There is no redundancy here.
    - Given A → B: Several tuples could have the same A value, and if so, they'll all have the same B value!

1NF

every attribute of a relation must be atomic.

在关系模式R中的每一个具体关系r中，如果每个属性值都是不可再分的最小数据单位，则称R是第一范式的关系

name	dept	address		
		prov	city	street

Non 1NF

name	dept	prov	city	street

1NF

2NF

- R∈1NF and ***no partially function dependency*** exists between attributes and keys.

如果关系模式R中的所有非主属性都完全依赖于任意一个候选关键字，则称关系R是属于第二范式的。

S(S#, SNAME, AGE, ADDR, C#, GRADE)

--- non 2NF

Problems of non 2NF:

- ***Insert abnormity***

  can not insert the students' information who have not selected course.

- ***Delete abnormity***

  if a student unselect all courses, his basic information is also lost.

- ***Hard to update***

  because of redundancy, it is hard to keep consistency when update.

Resolving:

According to the rule of "**one fact in one place**" to decompose the relation into 2 new relations:

S(S#, SNAME, AGE, ADDR)

SC(S#, C#, GRADE)

3NF

- R∈2NF and ***no transfer (transitive) function dependency*** exists between attributes.

如果关系模式R中的所有非主属性对任何候选关键字都不存在传递信赖，则称关系R是属于第三范式的。

EMP(EMP#, SAL_LEVEL, SALARY)

--- non 3NF

Third Normal Form (3NF)

- Relation R with FDs *F* is in 3NF if, for all X→ A in F+
  - A ∈ X (called a *trivial* FD), or
  - X contains a key for R, or
  - A is part of some key for R.
- If R is in 3NF, some redundancy is possible. It is a compromise, used when BCNF not achievable (e.g., no ``good'' decomp, or performance considerations).
  - *Lossless-join, dependency-preserving decomposition of R into a collection of 3NF relations always possible.*

Problems of non 3NF

- ***Insert abnormity***: before the employees's sal_level are decided, the correspondence between sal_level and salary can not input.
- ***Delete abnormity***: if some sal_level has only one man, the correspondence between sal_level and salary of this level will be lost when the man is deleted.
- ***Hard to update***: because of redundancy, it is hard to keep consistency when update.

Resolving:

According to the rule of "**one fact in one place**" to decompose the relation into 2 new relations:

EMP(EMP#,SAL_LEVEL)

SAL(SAL_LEVEL,SALARY)

Boyce-Codd Normal Form (BCNF)

- Relation R with FDs *F* is in BCNF if, for all X→ A in F+
  - A ∈ X (called a *trivial* FD), or
  - X contains a key for R.
- ***In other words, R is in BCNF if the only non-trivial FDs that hold over R are key constraints.***
  - No dependency in R that can be predicted using FDs alone.
  - If we are shown two tuples that agree upon the X value, we cannot infer the A value in one tuple from the A value in the other.

What Does 3NF Achieve?

- If 3NF violated by X → A, one of the following holds:
  - X is a subset of some key K
    - We store (X, A) pairs redundantly.
  - X is not a proper subset of any key.
    - There is a chain of FDs K→ X → A, which means that we cannot associate an X value with a K value unless we also associate an A value with an X value.
- But: even if relation is in 3NF, these problems could arise.
  - e.g., Reserves SBDC(sailor, boat, date, captain), S→ C, C → S is in 3NF, but for each reservation of sailor S, same (S, C) pair is stored.
- Thus, 3NF is indeed a compromise relative to BCNF.

Material Card

How to define relations to express the information on this card?

Equipment name： Type： Code：							
Unit price： Store place： room rack layer position							
date	voucher No.	coming/going place	take in	take out	balance	sum	remark

Decomposition of a Relation Scheme

- Suppose that relation R contains attributes *A1 ... An.* A *decomposition* of R consists of replacing R by two or more relations such that:
  - Each new relation scheme contains a subset of the attributes of R (and no attributes that do not appear in R), and
  - Every attribute of R appears as an attribute of one of the new relations.
- Intuitively, decomposing R means we will store instances of the relation schemes produced by the decomposition, instead of instances of R.
- E.g., Can decompose SNLRWH into SNLRH and RW.

Example Decomposition

Hourly_Emps (*ssn, name, lot, rating, hrly_wages, hrs_worked*)

- Decompositions should be used only when needed.
  - SNLRWH has FDs S → SNLRWH and R → W
  - Second FD causes violation of 3NF; W values repeatedly associated with R values. Easiest way to fix this is to create a relation RW to store these associations, and to remove W from the main schema:
  - i.e., we decompose SNLRWH into SNLRH and RW
- The information to be stored consists of SNLRWH tuples. If we just store the projections of these tuples onto SNLRH and RW, are there any potential problems that we should be aware of?

Problems with Decompositions

- There are three potential problems to consider:
  - ***Some queries become more expensive.***
    - e.g., How much did sailor Joe earn? (salary = W*H)
  - ***Given instances of the decomposed relations, we may not be able to reconstruct the corresponding instance of the original relation!***
  - ***Checking some dependencies may require joining the instances of the decomposed relations.***
- ***Tradeoff***: ***Must consider these issues vs. redundancy.***

Summary of Schema Refinement

- If a relation is in BCNF, it is free of redundancies that can be detected using FDs. Thus, trying to ensure that all relations are in BCNF is a good heuristic.
- If a relation is not in BCNF, we can try to decompose it into a collection of BCNF relations.
  - Must consider whether all FDs are preserved. If a lossless- join, dependency preserving decomposition into BCNF is not possible (or unsuitable, given typical queries), should consider decomposition into 3NF.
  - Decompositions should be carried out and/or re-examined while keeping *performance requirements* in mind.


SOUTHEAST UNIVERSITY
東南大學
1902
南京

Features of Good Relational Designs

- Suppose we combine *instructor* and *department* into *in_dep,* which represents the natural join on the relations *instructor* and *department*

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- There is repetition of information
- Need to use null values (if we add a new department with no instructors)

Decomposition

- The only way to avoid the repetition-of-information problem in the *in_dep* schema is to decompose it into two schemas – instructor and *department* schemas.
- Not all decompositions are good. Suppose we decompose
  employee(ID, name, street, city, salary)

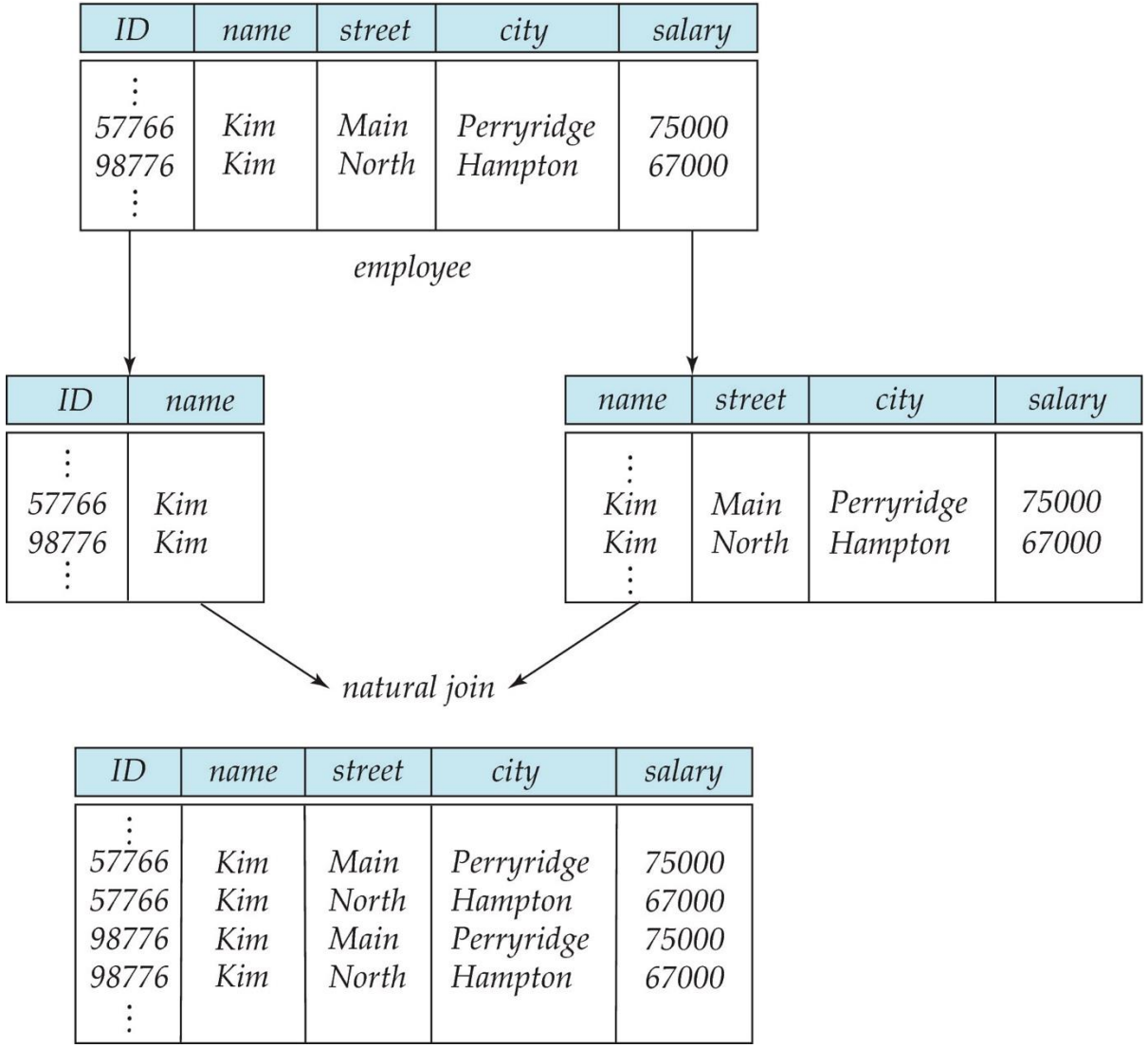
  into

  employee1 (*ID*, *name*)

  employee2 (*name*, *street, city, salary*)

  The problem arises when we have two employees with the same name
- The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a ***lossy decomposition.***

A Lossy Decomposition



Example of Lossless Decomposition

- Decomposition of *R = (A, B, C)*

  $R_1 = (A, B)$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A (r) \bowtie \Pi_B (r)$

A	B	C
α	1	A
β	2	B

Redundancy in 3NF

- Consider the schema R below, which is in 3NF
  - $R = (J, K, L)$
  - $F = \{JK \rightarrow L, L \rightarrow K\}$
  - And an instance table:

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

- What is wrong with the table?
  - Repetition of information
  - Need to use null values (e.g., to represent the relationship l_2, k_2 where there is no corresponding value for J)

Comparison of BCNF and 3NF

- Advantages to 3NF over BCNF. It is always possible to obtain a 3NF design without sacrificing losslessness or dependency preservation.
- Disadvantages to 3NF.
  - We may have to use null values to represent some of the possible meaningful relationships among data items.
  - There is the problem of repetition of information.

How good is BCNF?

- There are database schemas in BCNF that do not seem to be sufficiently normalized
- Consider a relation

  inst_info (ID, child_name, phone)

  - where an instructor may have more than one phone and can have multiple children
  - Instance of *inst_info*

ID	*child_name*	*phone*
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	William	512-555-4321

- There are no non-trivial functional dependencies and therefore the relation is in BCNF
- Insertion anomalies – i.e., if we add a phone 981-992-3443 to 99999, we need to add two tuples

  (99999, David, 981-992-3443)
  (99999, William, 981-992-3443)

Higher Normal Forms

- It is better to decompose *inst_info* into:
  - *inst_child*:

ID	*child_name*
99999	David
99999	William

  - *inst_phone:*

ID	*phone*
99999	512-555-1234
99999	512-555-4321

- This suggests the need for higher normal forms, such as Fourth Normal Form (4NF), which we shall see later


SOUTHEAST UNIVERSITY
東南大學
1902
南京

Multivalued Dependencies (MVDs)

- Suppose we record names of children, and phone numbers for instructors:
  - *inst_child*(*ID*, *child_name*)
  - *inst_phone*(*ID*, *phone_number*)
- If we were to combine these schemas to get
  - *inst_info*(*ID*, *child_name*, *phone_number*)
  - Example data:
    (99999, David, 512-555-1234)
    (99999, David, 512-555-4321)
    (99999, William, 512-555-1234)
    (99999, William, 512-555-4321)
- This relation is in BCNF

MVD -- Tabular representation

- Tabular representation of $\alpha \rightarrow\rightarrow \beta$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \ldots a_i$	$a_{i+1} \ldots a_j$	$a_{j+1} \ldots a_n$
t_2	$a_1 \ldots a_i$	$b_{i+1} \ldots b_j$	$b_{j+1} \ldots b_n$
t_3	$a_1 \ldots a_i$	$a_{i+1} \ldots a_j$	$b_{j+1} \ldots b_n$
t_4	$a_1 \ldots a_i$	$b_{i+1} \ldots b_j$	$a_{j+1} \ldots a_n$

Example

- In our example:

$$ID \twoheadrightarrow child\_name$$
$$ID \twoheadrightarrow phone\_number$$

- The above formal definition is supposed to formalize the notion that given a particular value of *Y* (*ID*) it has associated with it a set of values of *Z (child_name)* and a set of values of *W (phone_number)*, and these two sets are in some sense independent of each other.
- Note:
  - If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$
  - Indeed we have (in above notation) $Z_1 = Z_2$
    The claim follows.

*Fourth Normal Form

- A relation schema *R* is in **4NF** with respect to a set *D* of functional and multivalued dependencies if for all multivalued dependencies in D^+ of the form $\alpha \rightarrow\rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, ***at least one*** of the following hold:
  - $\alpha \rightarrow\rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$ or $\alpha \cup \beta = R$)
  - α is a superkey for schema *R*
- If a relation is in 4NF it is in BCNF

- **ER Model and ER Diagram**
- **Database Design Method**

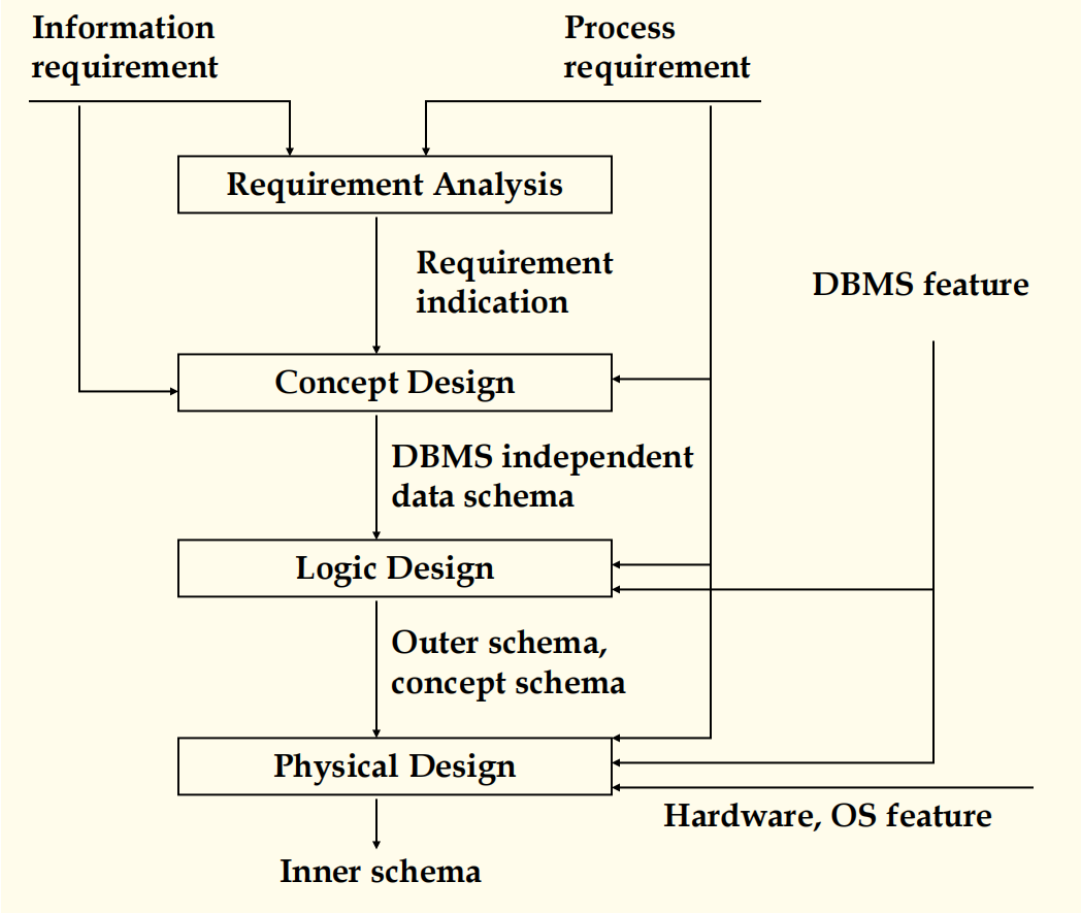
Database Design Method

- **Procedure oriented method**
- This method takes business procedures as center, the database schema is designed basically in accordance directly with the vouchers, receipts, reports, etc. in business. Because of no detailed analysis on data and inner relationships between data, although it is fast at the beginning of the project, it is hard to ensure software quality and the system will be hard to fit future changes in requirement and environment. So this method is not suitable for the development of a large, complex system.
- **Data oriented method**
- This method design the database schema based on the detailed analysis on data and inner relationships between data which are involved in business procedures. It takes data as center, not procedures. It can not only fulfill the current requirements, but also some potential requirements. It is liable to fit future changes in requirement and environment. It is recommended in the development of large, complex systems

Steps in Database Design

- **Requirements Analysis**
  - user needs; what must database do?
- **Conceptual Design**
  - *high level description (often done w/ER model)*
  - Object-Relational Mappings (ORMs: Hibernate, Rails, Django, etc) encourage you to program here
- **Logical Design**
  - translate ER into DBMS data model
  - ORMs often require you to help here too
- **Schema Refinement**
  - consistency, normalization
- **Physical Design** - indexes, disk layout
- **Security Design** - who accesses what, and how

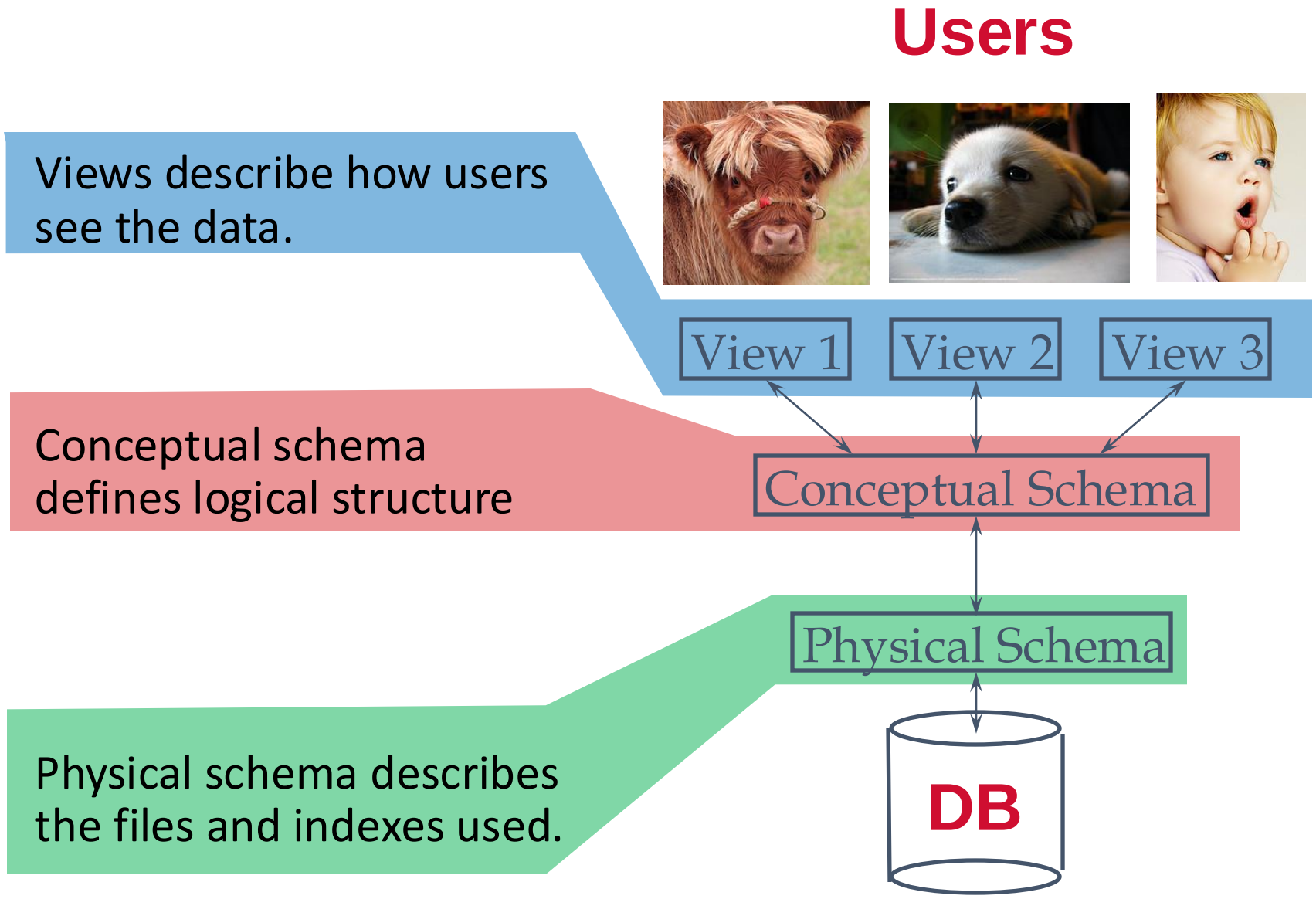
Database Design Flow



Describing Data: Data Models

- **Data model :** collection of concepts for describing data.
- **Schema:** description of a particular collection of data, using a given data model.
- **Relational model of data**
  - Main concept: relation (table), rows and columns
  - Every relation has a schema
    - describes the columns
    - column names and domains

Levels of Abstraction



Example: University Database

- **Conceptual schema:**
  - `Students(sid text, name text, login text, age integer, gpa float)`
  - `Courses(cid text, cname text, credits integer)`
  - `Enrolled(sid text, cid text, grade text)`
- **Physical schema:**
  - Relations stored as unordered files.
  - Index on first column of Students.
- **External Schema** (View):
  - Course_info(cid text, enrollment integer)

Data Independence

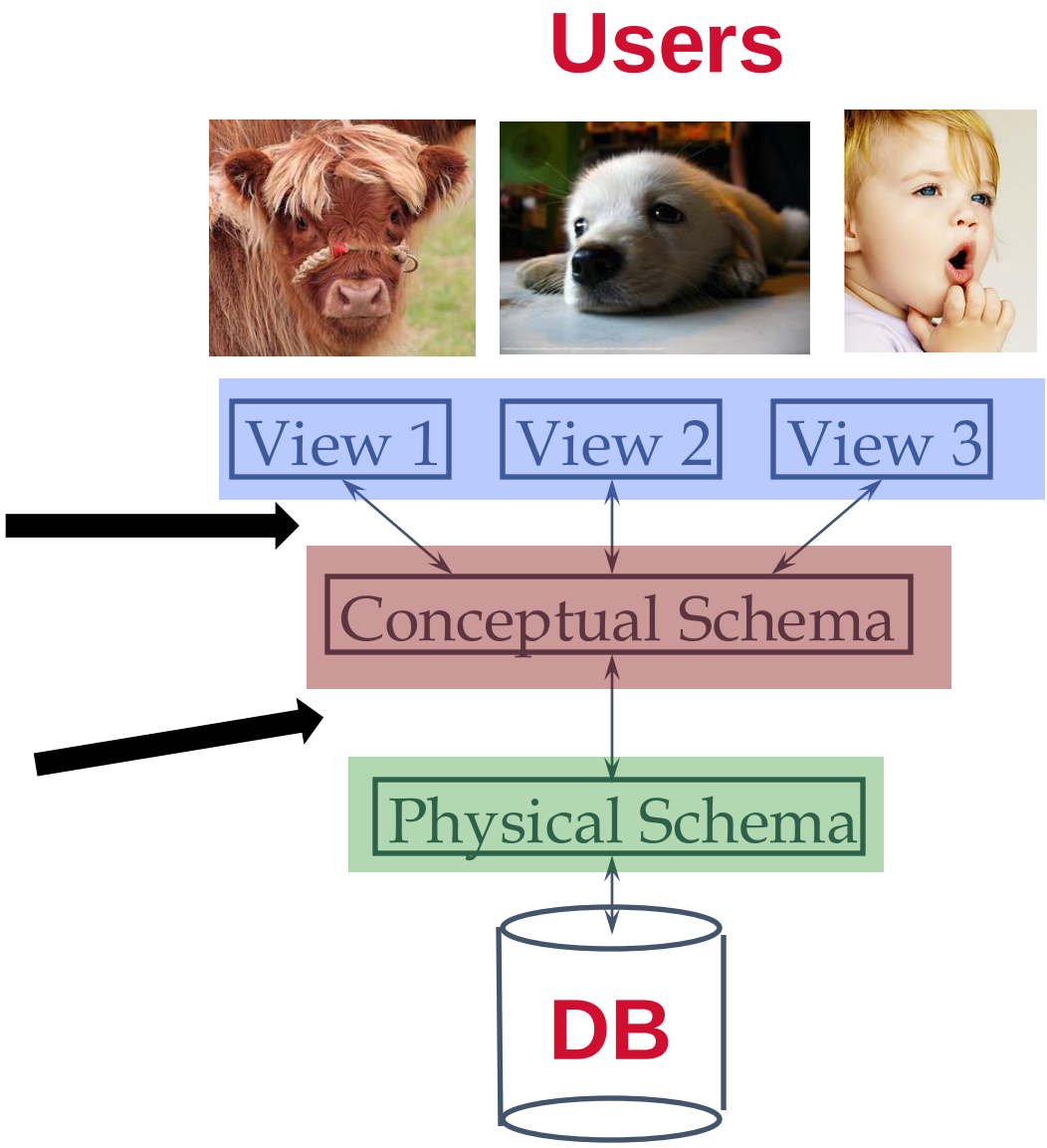
- Insulate apps from structure of data

- **Logical data independence:**
  - Maintain views when logical structure changes
- **Physical data independence:**
  - Maintain logical structure when physical structure changes

Levels of Abstraction, cont

Logical data independence

Physical data independence



Data Independence, cont

- Insulate apps from structure of data

- **Logical data independence:**
  - Maintain views when logical structure changes
- **Physical data independence:**
  - Maintain logical structure when physical structure changes

- Q: Why particularly important for DBMS?
  - Because databases and their associated applications persist


SOUTHEAST UNIVERSITY
東南大學
1902
南京

Requirement Analysis

- A very important part of system requirement analysis. In requirement analysis phase, the data dictionary and DFD (or UML) diagrams are the most important to database design.
- **Dictionary and DFD**
  - Name conflicts
  - Homonym(the same name with different meanings)
  - Synonym(the same meaning in different names)
  - Concept conflicts
  - Domain conflicts
- **About coding**
  - Standardization of information
  - Identifying entities
  - Compressing information
- **Through requirement analysis, all information must be with unique source and unique responsibility.**

Concept Design

- Based on data dictionary and DFD, analyze and classify the data in data dictionary, and refer to the processing requirement reflected in DFD, identify entities, attributes, and relationships between entities. Then we can get concept schema of the database.
  - Identify Entities
  - Define the relationships between entities
  - Draw ER diagram and discuss it with user
- It is proposed to use ER design tools such as ERWin, Rose, etc.


SOUTHEAST UNIVERSITY
東南大學
1902
南京

Entity-Relationship Model

- Relational model is a great formalism
  - But a bit detailed for design time
  - Too fussy for brainstorming
  - Hard to communicate to “customers”
- Entity-Relationship model: a graph-based model
  - can be viewed as a graph, or a veneer over relations
    - “feels” more flexible, less structured
  - corresponds well to “Object-Relational Mapping”
    - (ORM) SW packages
    - Ruby-on-Rails, Django, Hibernate, Sequelize, etc.

Steps in Database Design, again

- Requirements Analysis
  - user needs; what must database do?
- **Conceptual Design**
  - ***high level description (often done w/ER model)***
  - **ORM encourages you to program here**
- Logical Design
  - translate ER into DBMS data model
  - ORMs often require you to help here too
- Schema Refinement
  - consistency, normalization
- Physical Design - indexes, disk layout
- Security Design - who accesses what, and how

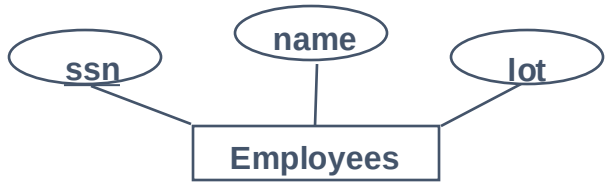
Conceptual Design

- What are the entities and relationships?
  - And what info about E's & R's should be in DB?
- What integrity constraints ("business rules") hold?

- ER diagram is the "schema"
- Can map an ER diagram into a relational schema.

- Conceptual design is where the data engineering begins

ER Model Basics: Entities



- An **entity** is an object that exists and is distinguishable from other objects.
  - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
  - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
  - Example:

    instructor = (*ID, name, salary*)
    course= (*course_id, title, credits*)
- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.

Entity Sets -- *instructor* and *student*

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

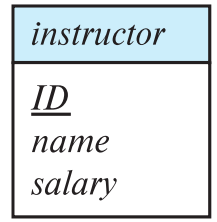
instructor

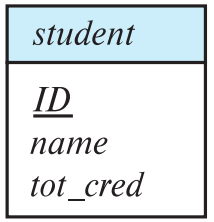
98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

Representing Entity sets in ER Diagram

- Entity sets can be represented graphically as follows:
  - Rectangles represent entity sets.
  - Attributes listed inside entity rectangle
  - Underline indicates primary key attributes





Relationship Sets

- A **relationship** is an association among several entities

  Example:

  44553 (Peltier) — *advisor* — 22222 (Einstein)

  student entity — relationship set — *instructor* entity

- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \ldots e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

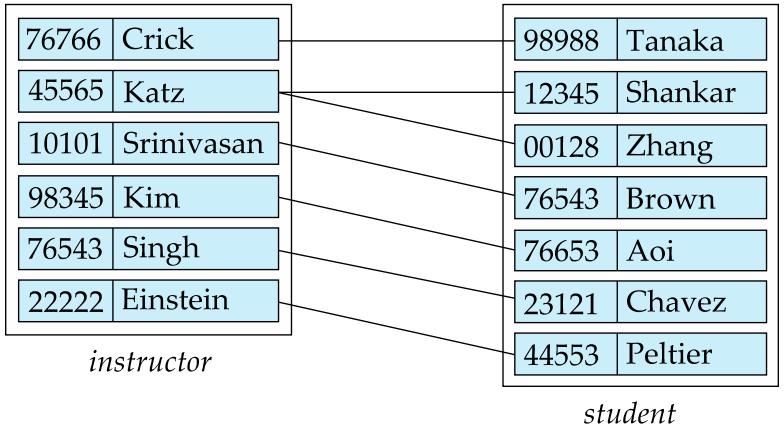
  where $(e_1, e_2, \ldots, e_n)$ is a relationship

  - Example:

    $(44553,22222) \in advisor$

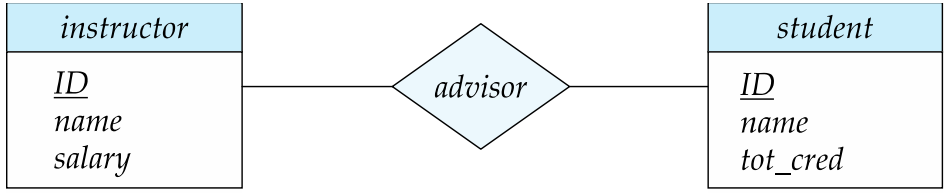
Relationship Sets (Cont.)

- Example: we define the relationship set *advisor* to denote the associations between students and the instructors who act as their advisors.
- Pictorially, we draw a line between related entities.



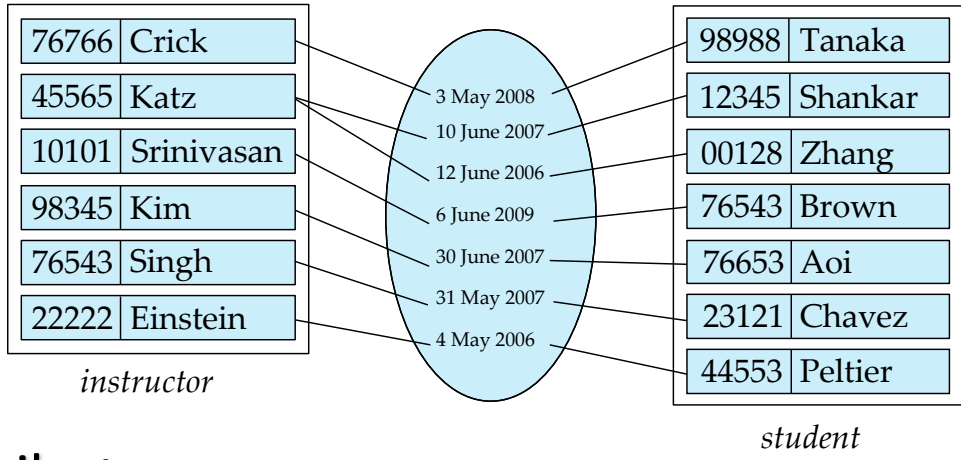
Representing Relationship Sets via ER Diagrams

- Diamonds represent relationship sets.

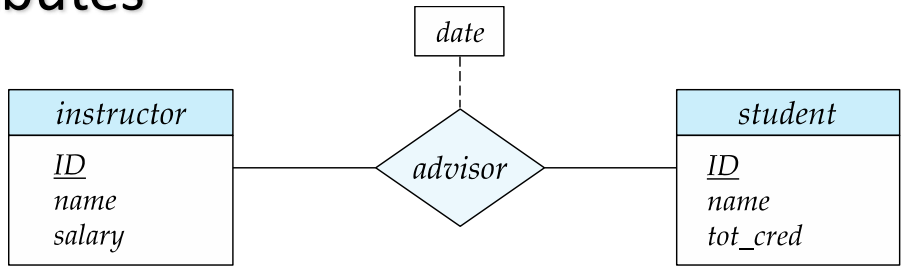


Relationship Sets (Cont.)

- An attribute can also be associated with a relationship set.
- For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor

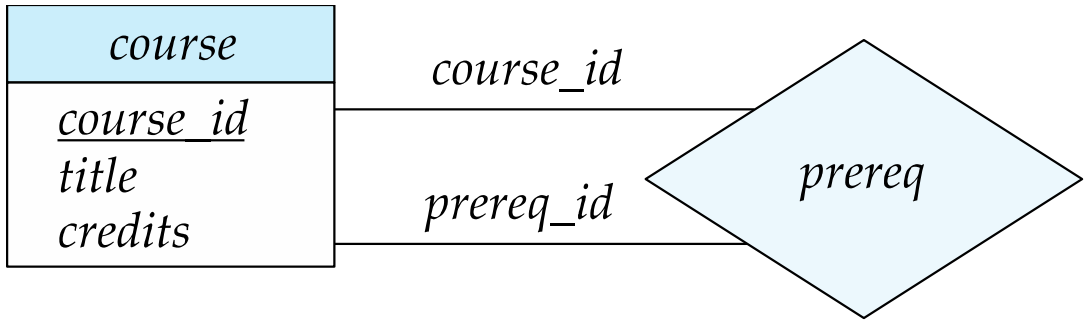


Relationship Sets with Attributes



Roles

- Entity sets of a relationship need not be distinct
  - Each occurrence of an entity set plays a “role” in the relationship
- The labels “*course_id*” and “*prereq_id*” are called **roles**.



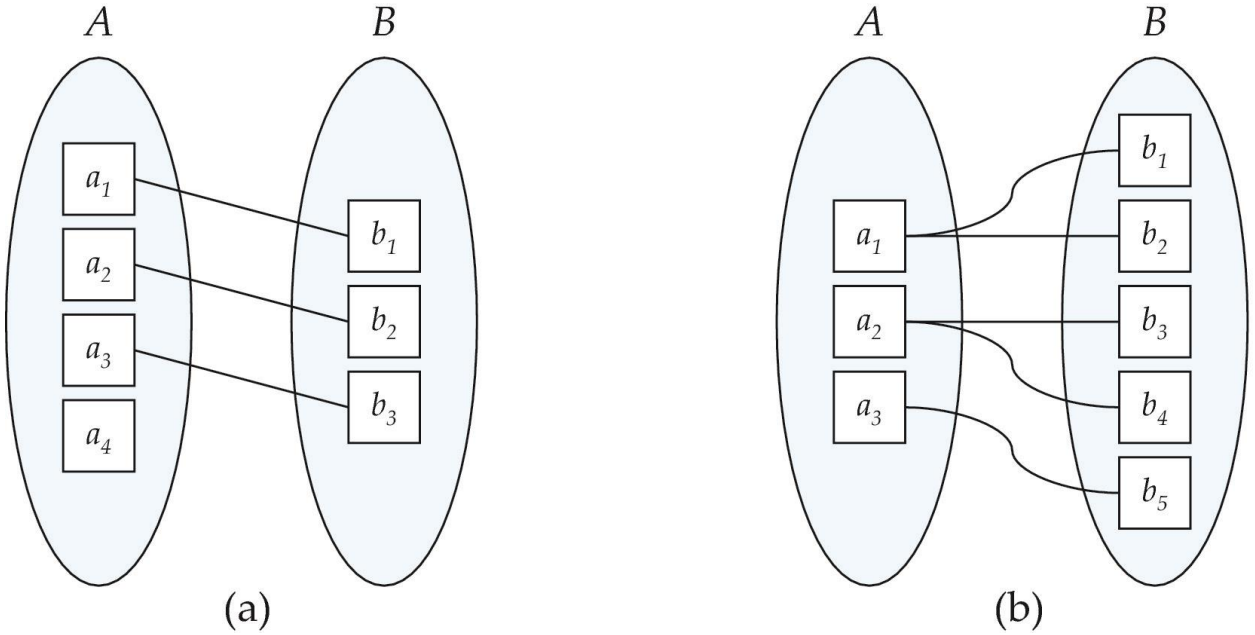
Degree of a Relationship Set

- Binary relationship
  - involve two entity sets (or degree two).
  - most relationship sets in a database system are binary.
- Relationships between more than two entity sets are rare. Most relationships are binary. (More on this later.)
  - Example: *students* work on research *projects* under the guidance of an *instructor*.
  - relationship *proj_guide* is a ternary relationship between *instructor, student,* and *project*

Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
  - One to one
  - One to many
  - Many to one
  - Many to many

Mapping Cardinalities

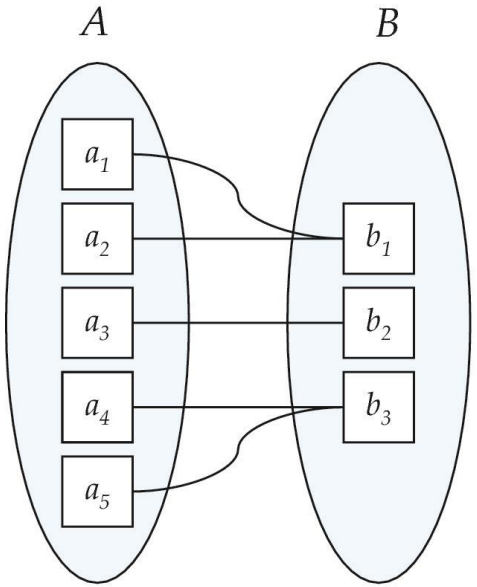


One to one

One to many

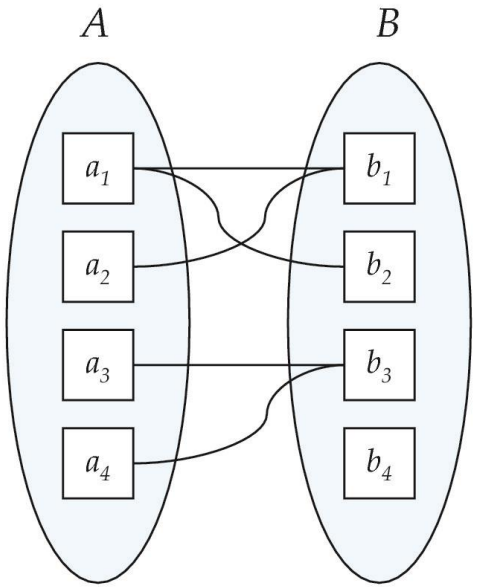
Note: Some elements in *A* and *B* may not be mapped to any elements in the other set

Mapping Cardinalities



(a)

Many to one



(b)

Many to many

Note: Some elements in A and B may not be mapped to any elements in the other set

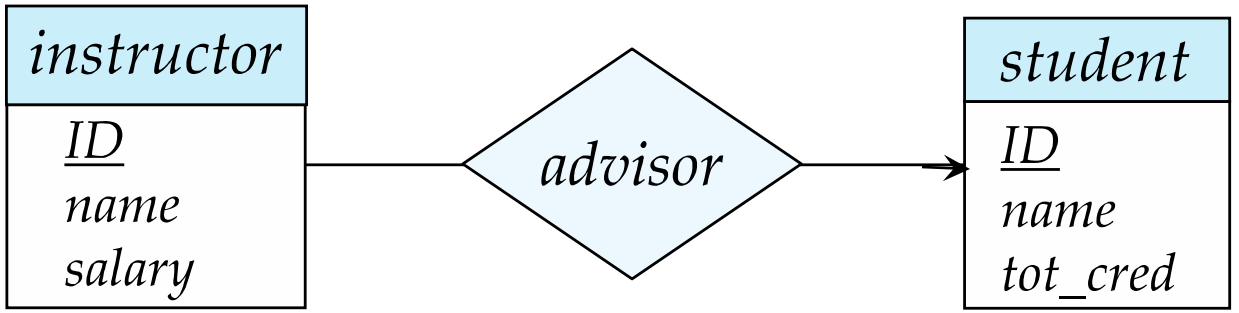
Representing Cardinality Constraints in ER Diagram

- We express cardinality constraints by drawing either a directed line (→), signifying "one," or an undirected line (—), signifying "many," between the relationship set and the entity set.

- One-to-one relationship between an *instructor* and a *student* :
  - A student is associated with at most one *instructor* via the relationship *advisor*
  - A *student* is associated with at most one *department* via *stud_dept*

Relationships

- In a many-to-one relationship between an *instructor* and a *student,*
  - an instructor is associated with at most one student via *advisor*,
  - and a student is associated with several (including 0) instructors via *advisor*

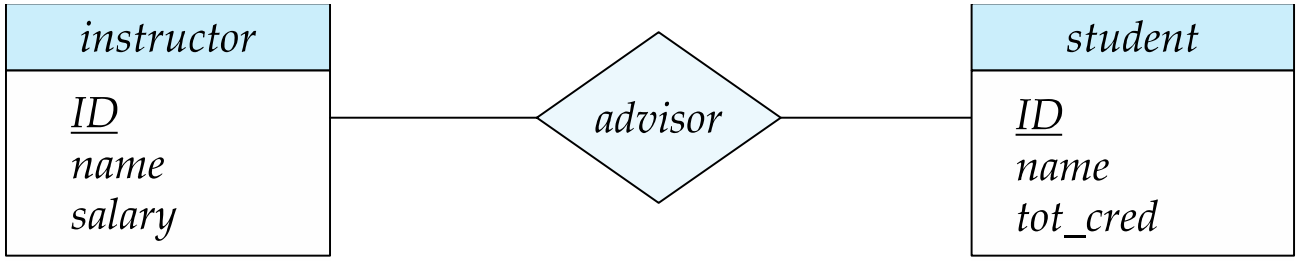


One-to-Many Relationship

- one-to-many relationship between an *instructor* and a *student*
  - an instructor is associated with several (including 0) students via *advisor*
  - a student is associated with at most one instructor via advisor,

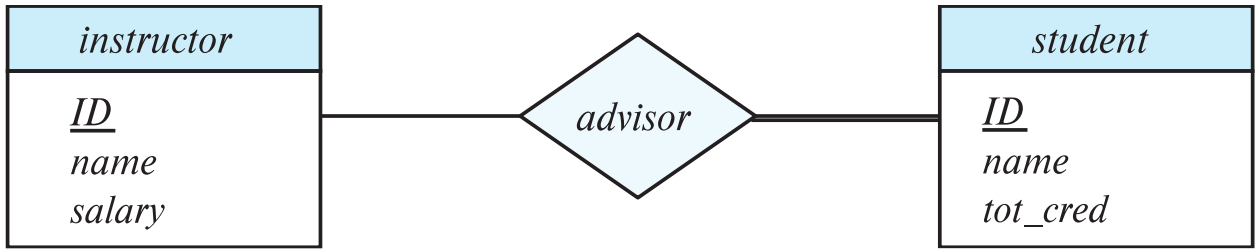
Many-to-Many Relationship

- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*



Total and Partial Participation

- **Total participation** (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set



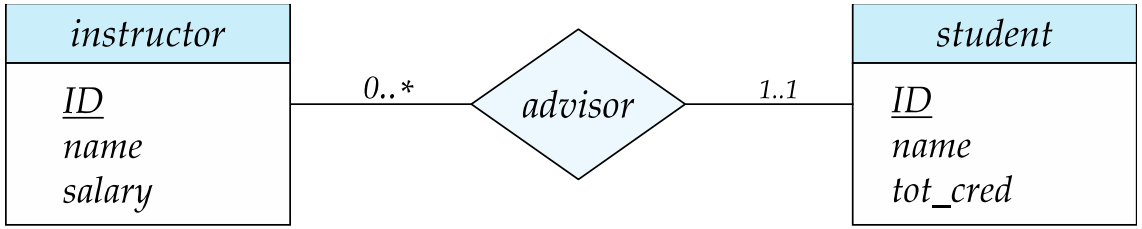
participation of *student* in *advisor* relation is total

  - every *student* must have an associated instructor

- **Partial participation**: some entities may not participate in any relationship in the relationship set
  - Example: participation of *instructor* in *advisor* is partial

Notation for Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form *l..h*, where *l* is the minimum and *h* the maximum cardinality
  - A minimum value of 1 indicates total participation.
  - A maximum value of 1 indicates that the entity participates in at most one relationship
  - A maximum value of * indicates no limit.
- Example



  - Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors

Primary Key

- Primary keys provide a way to specify how entities and relations are distinguished. We will consider:
  - Entity sets
  - Relationship sets.
  - Weak entity sets

- By definition, individual entities are distinct.
- From database perspective, the differences among them must be expressed in terms of their attributes.
- The values of the attribute values of an entity must be such that they can uniquely identify the entity.
  - No two entities in an entity set are allowed to have exactly the same value for all attributes.
- A key for an entity is a set of attributes that suffice to distinguish entities from each other

Primary Key for Relationship Sets

- To distinguish among the various relationships of a relationship set we use the individual primary keys of the entities in the relationship set.
  - Let *R* be a relationship set involving entity sets E1, E2, .. En
  - The primary key for R is consists of the union of the primary keys of entity sets E1, E2, ..En
  - If the relationship set *R* has attributes a1, a2, .., am associated with it, then the primary key of *R* also includes the attributes a1, a2, .., am
- Example: relationship set “advisor”.
  - The primary key consists of *instructor.ID* and s*tudent.ID*
- The choice of the primary key for a relationship set depends on the mapping cardinality of the relationship set.

Weak Entity Sets

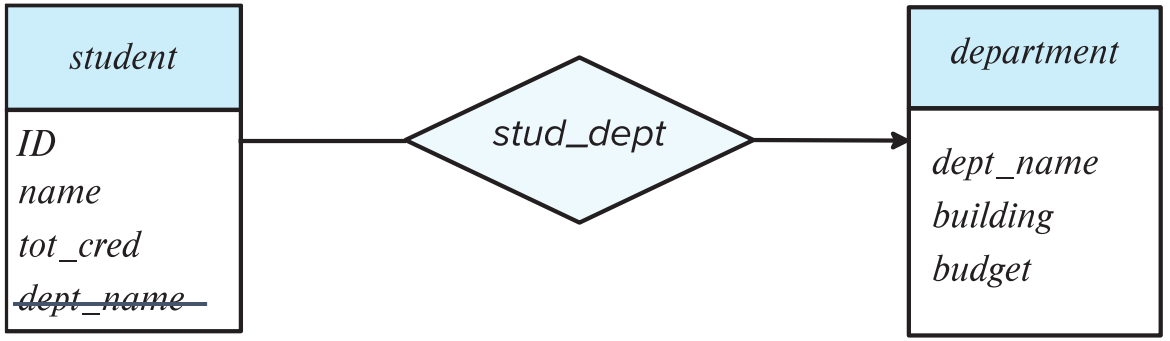
- Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set.
- The identifying entity set is said to **own** the weak entity set that it identifies.
- The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.

Expressing Weak Entity Sets

- In E-R diagrams, a weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.
- Primary key for *section* – (*course_id, sec_id, semester, year*)

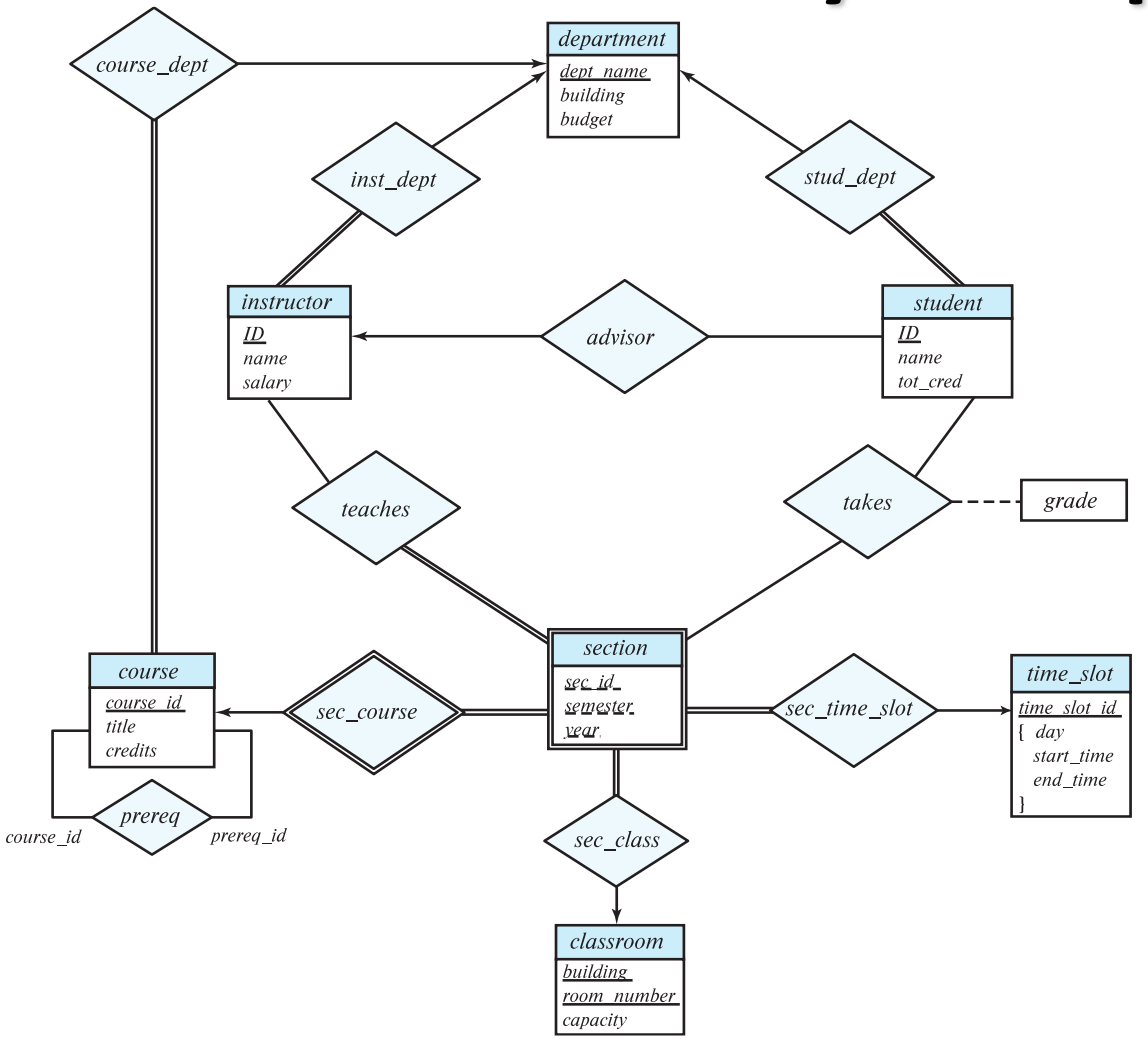
Redundant Attributes

- Suppose we have entity sets:
  - *student*, with attributes: *ID*, *name, tot_cred*, *dept_name*
  - *department,* with attributes: *dept_name, building, budget*
- We model the fact that each student has an associated department using a relationship set *stud_dept*
- The attribute *dept_name* in *student* below replicates information present in the relationship and is therefore redundant
  - and needs to be removed.
- BUT: when converting back to tables, in some cases the attribute gets reintroduced



(a) Incorrect use of attribute

E-R Diagram for a University Enterprise

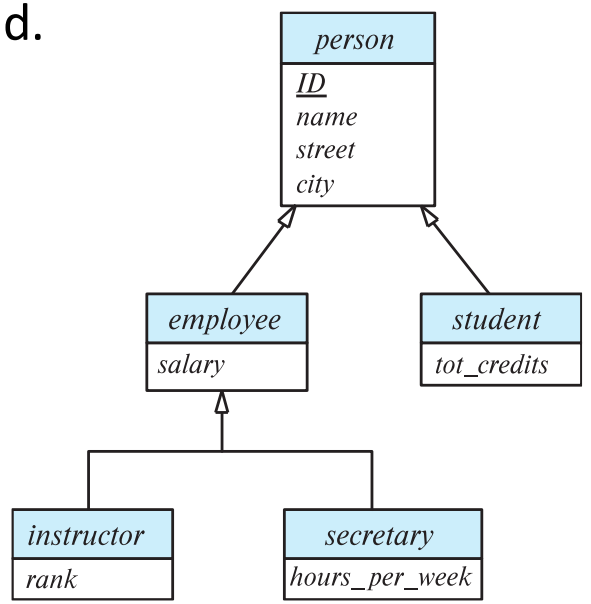


Extended E-R Features

Specialization

- Top-down design process; we designate sub-groupings within an entity set that are distinctive from other entities in the set.
- These sub-groupings become lower-level entity sets that have attributes or participate in relationships that do not apply to the higher-level entity set.
- Depicted by a *triangle* component labeled ISA (e.g., *instructor* "is a" *person*).
- **Attribute inheritance** – a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.

Overlapping – *employee* and *student*
Disjoint – *instructor* and *secretary*
Total and partial

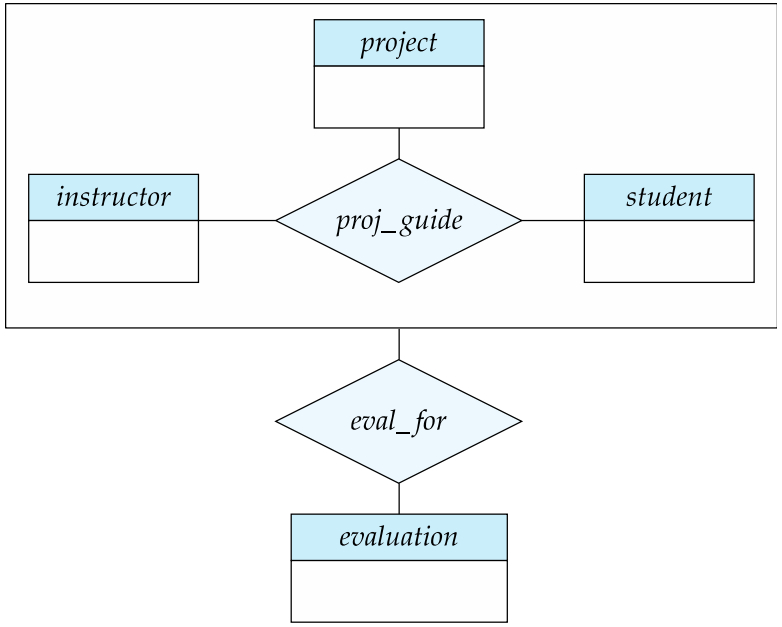


Generalization

- **A bottom-up design process** – combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.

Aggregation

- Eliminate this redundancy via *aggregation* without introducing redundancy, the following diagram represents:
  - A student is guided by a particular instructor on a particular project
  - A student, instructor, project combination may have an associated evaluation



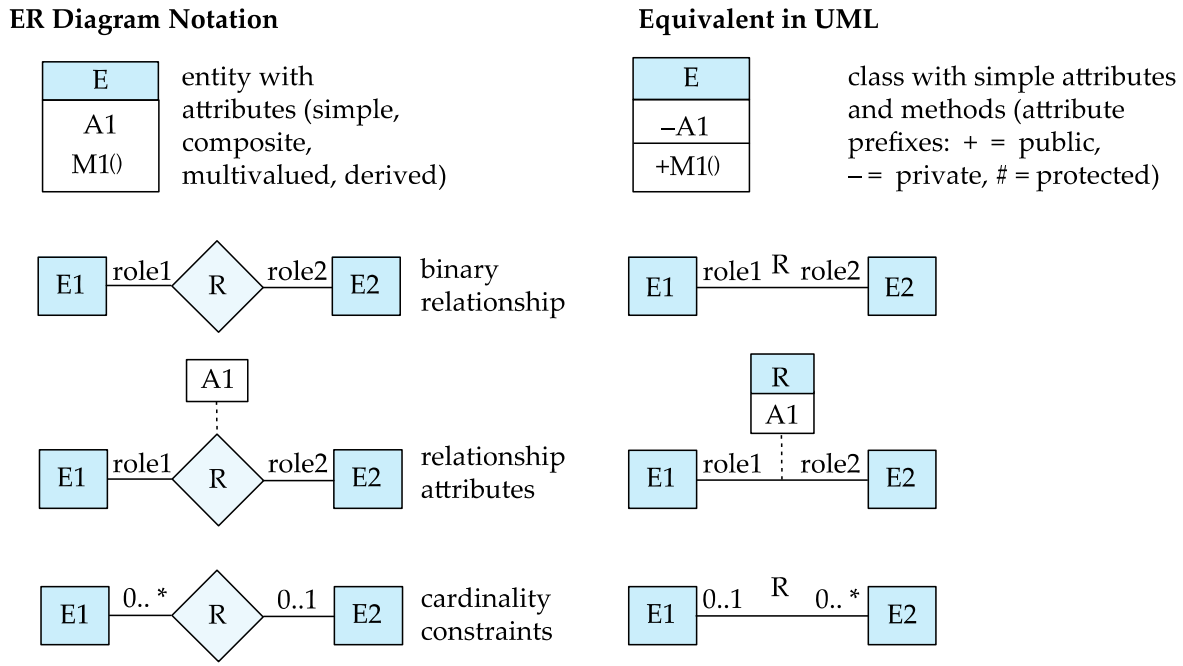
E-R Design Decisions

- The use of an attribute or entity set to represent an object.
- Whether a real-world concept is best expressed by an entity set or a relationship set.
- The use of a ternary relationship versus a pair of binary relationships.
- The use of a strong or weak entity set.
- The use of specialization/generalization – contributes to modularity in the design.
- The use of aggregation – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.

UML

- **UML**: Unified Modeling Language
- UML has many components to graphically model different aspects of an entire software system
- UML Class Diagrams correspond to E-R Diagram, but several differences.

ER vs. UML Class Diagrams



* Note reversal of position in cardinality constraint depiction

Logic Design

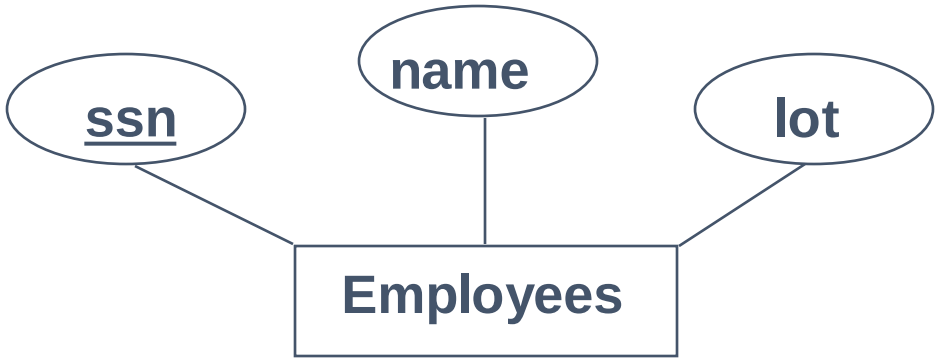
- According to the entities and relationships in ER diagram, define tables and views in target DBMS. Basic standard is 3NF.
  - Translate entities and relationships in ER diagram to tables
  - Naming rule of table and attribute
  - Define the type and domain of every attribute
  - Suitable denormalization
  - Necessary view
  - Consider the tables in legacy system
  - Interface tables

Converting ER to Relational

- Fairly analogous structure
- But many simple concepts in ER are subtle to specify in relations

Logical DB Design: ER to Relational

- Entity sets to tables. Easy.



ssn	name	lot
123-22-3666	Attishoo	48
231-31-5368	Smiley	22
131-24-3650	Smethurst	35

```
CREATE TABLE Employees
  (ssn CHAR(11),
   name CHAR(20),
   lot  INTEGER,
   PRIMARY KEY  (ssn))
```

Relationship Sets to Tables

In translating a **many-to-many** relationship set to a relation, attributes of the relation must include:

1) Keys for each participating entity set (as foreign keys). This set of attributes forms a ***superkey*** for the relation.

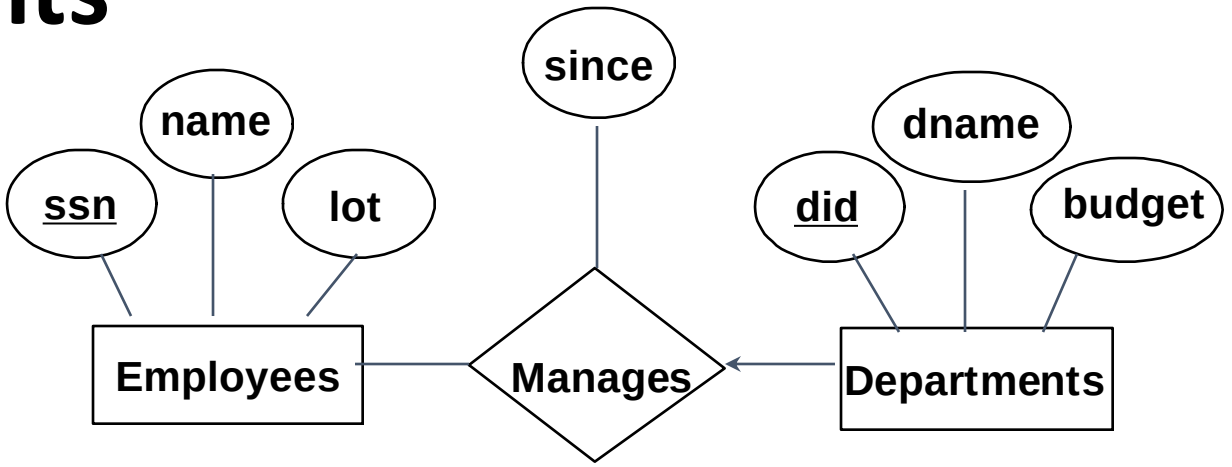
2) All descriptive attributes.

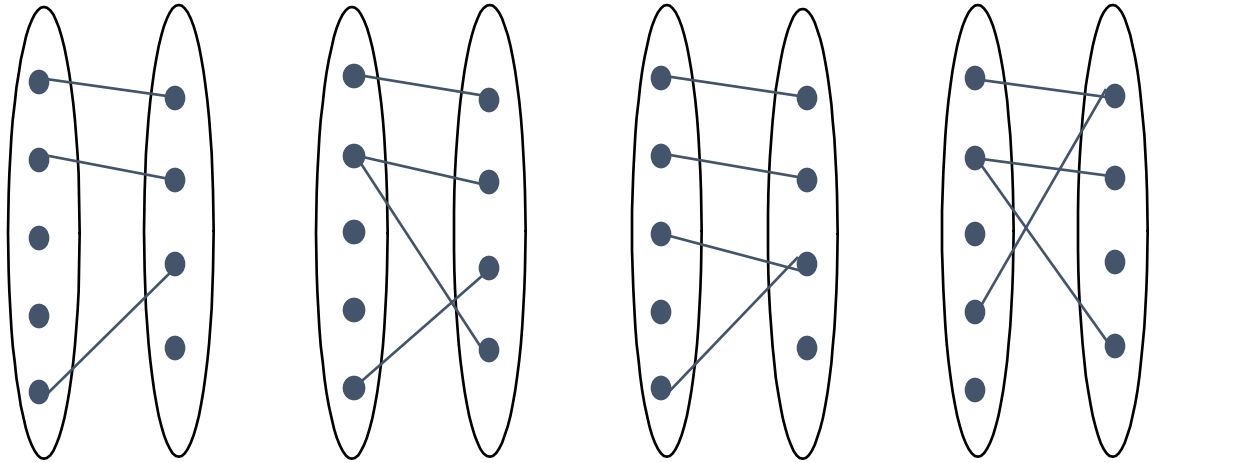
ssn	did	since
123-22-3666	51	1/1/91
123-22-3666	56	3/3/93
231-31-5368	51	2/2/92

```
CREATE TABLE Works_In(
  ssn  CHAR(1),
  did  INTEGER,
  since  DATE,
  PRIMARY KEY (ssn, did),
  FOREIGN KEY (ssn)
    REFERENCES Employees,
  FOREIGN KEY (did)
    REFERENCES Departments)
```

Review: Key Constraints

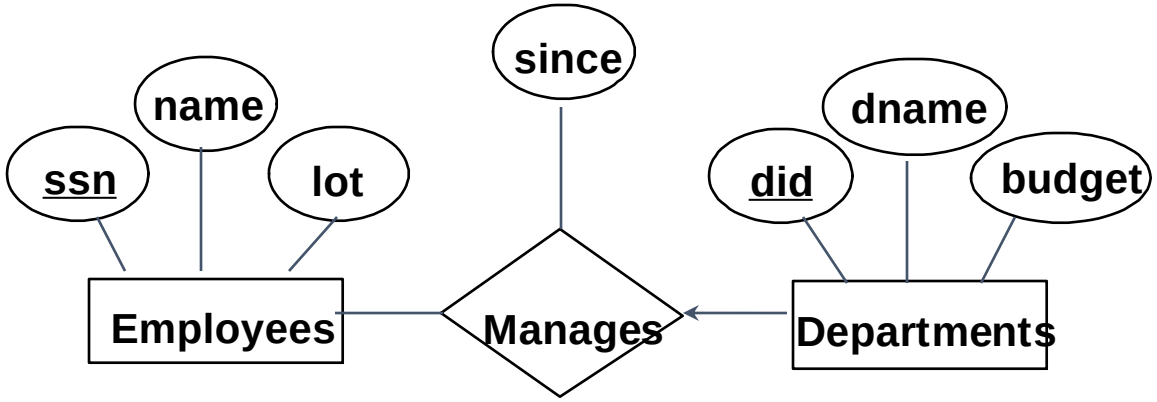
Each dept has at most one manager, according to the **key constraint** on Manages.





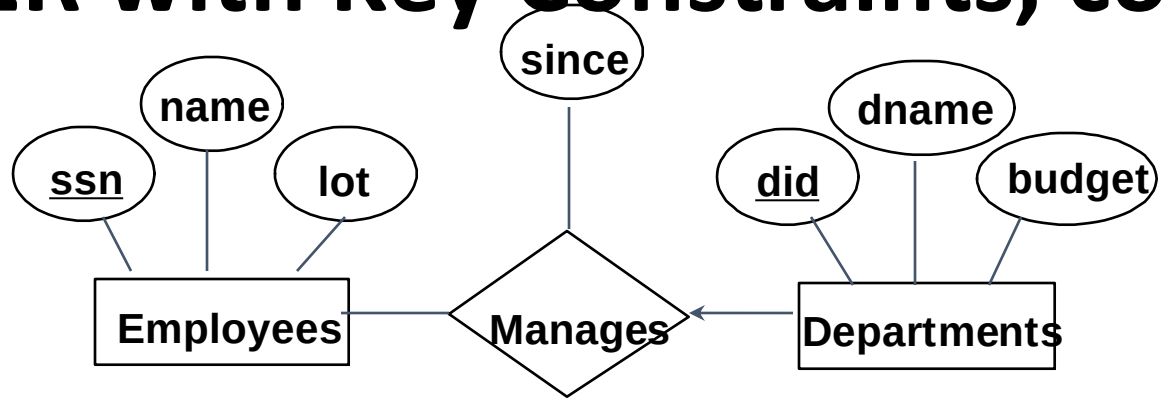
1-to-1 1-to Many Many-to-1 Many-to-Many

Translating ER with Key Constraints




```
CREATE TABLE  Manages(
 ssn  CHAR(11),
 did  INTEGER,
 since  DATE,
 PRIMARY KEY  (did),
 FOREIGN KEY (ssn)
   REFERENCES Employees,
 FOREIGN KEY (did)
   REFERENCES Departments)
```

Translating ER with Key Constraints, cont




```
CREATE TABLE  Manages(
 ssn  CHAR(11),
 did  INTEGER,
 since  DATE,
 PRIMARY KEY  (did),
 FOREIGN KEY (ssn)
   REFERENCES Employees,
 FOREIGN KEY (did)
   REFERENCES Departments)
```

Vs.

```
CREATE TABLE  Dept_Mgr(
 did  INTEGER,
 dname  CHAR(20),
 budget  REAL,
 ssn  CHAR(11),
 since  DATE,
 PRIMARY KEY  (did),
 FOREIGN KEY (ssn)
   REFERENCES Employees)
```

- Since each department has a unique manager, we could instead combine Manages and Departments.

Translating Weak Entity Sets

- Weak entity set and identifying relationship set are translated into a single table.
  - **When the owner entity is deleted, all owned weak entities must also be deleted.**

```
CREATE TABLE  Dep_Policy (
   pname  CHAR(20),
   age  INTEGER,
   cost  REAL,
   ssn  CHAR(11) NOT NULL,
   PRIMARY KEY  (pname, ssn),
   FOREIGN KEY  (ssn) REFERENCES Employees
      ON DELETE CASCADE)
```

Summary of Conceptual Design

- **Conceptual design** follows requirements analysis
  - Yields a high-level description of data to be stored
- ER model popular for conceptual design
  - Constructs are expressive, close to the way we think about applications.
  - Note: There are many variations on ER model
    - Both graphically and conceptually
- Basic constructs: **entities**, **relationships**, and **attributes** (of entities and relationships).
- Some additional constructs**: weak entities**, ISA hierarchies (see text if you're curious), and aggregation.

Summary of ER (Cont.)

- Basic integrity constraints
  - **key constraints**
  - **participation constraints**
- Some **foreign key** constraints are also implicit in the definition of a relationship set.
- Many other constraints (notably, **functional dependencies**) cannot be expressed.
- Constraints play an important role in determining the best database design for an enterprise.

Summary of ER (Cont….)

- ER design is **subjective**. Many ways to model a given scenario!
- Analyzing alternatives can be tricky! Common choices include:
  - Entity vs. attribute, entity vs. relationship, binary or n-ary relationship, whether or not to use aggregation
- For good DB design: resulting relational schema should be analyzed and refined further.
  - Functional Dependency information
    + normalization coming in subsequent lecture.


SOUTHEAST UNIVERSITY
東南大學
1902
南京

Physical Design

- For relational database, the main task in this phase is to consider creating necessary indexes according to the processing requirements, including single attribute indexes, multi attributes indexes, cluster indexes, etc. Generally, the attribute often as query conditions should have index.
- **Other problems:**
  - Partition design
  - Stored procedure
  - Trigger
  - Integrity constraints